SLIIT
Discover Your Future

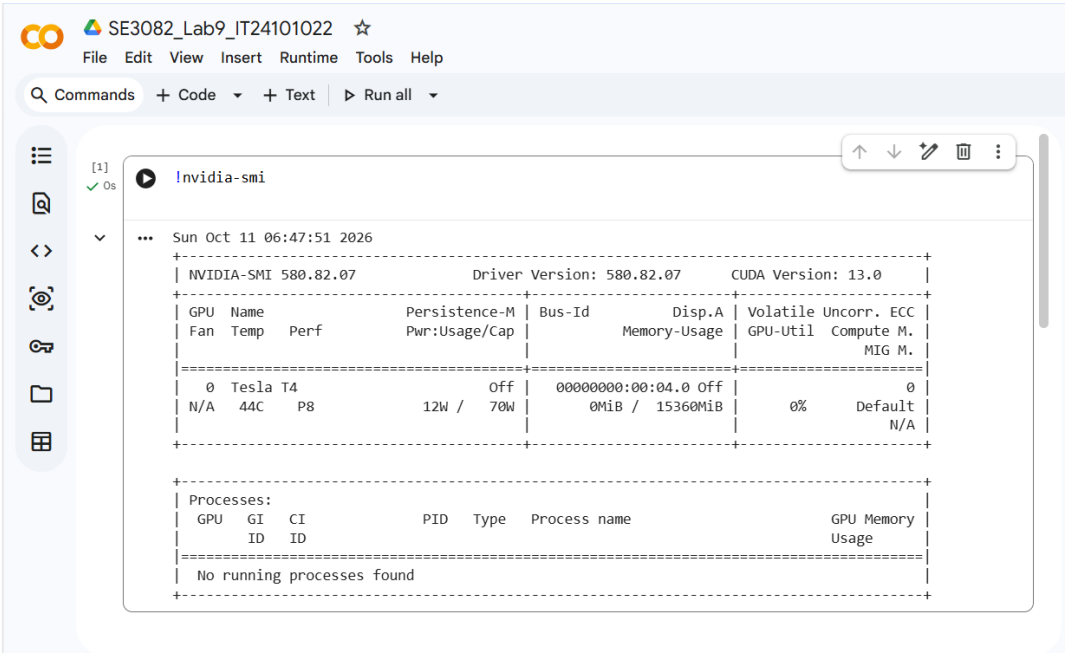
BSc (Hons) in Information Technology
Year 3

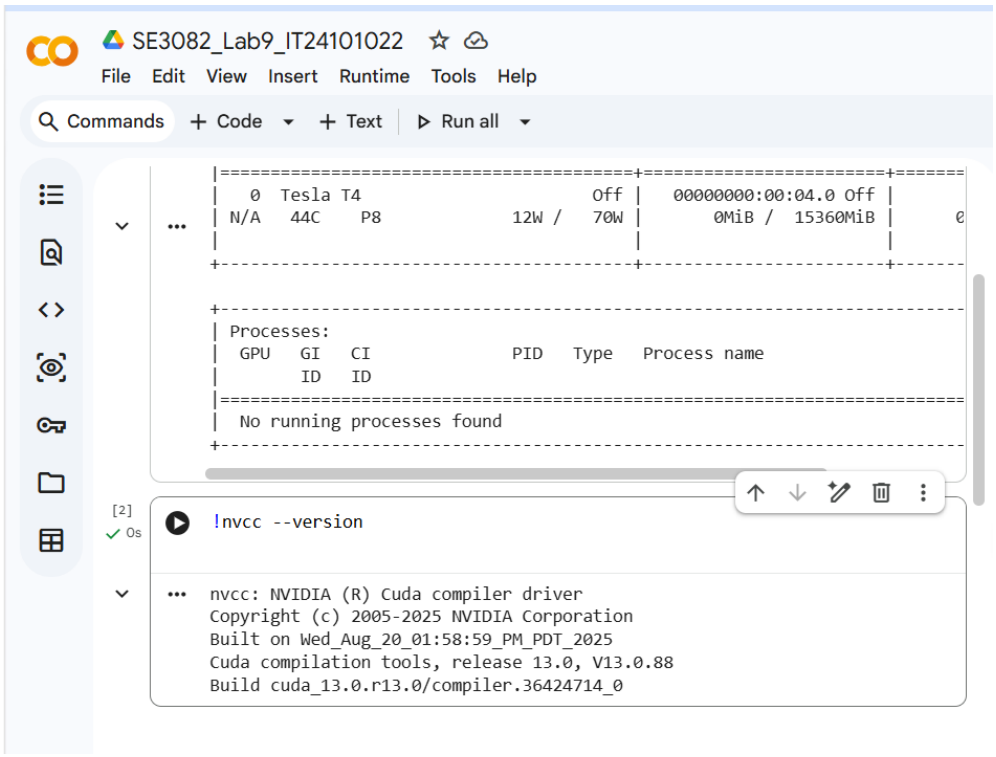
Lab Sheet 9 – Introduction to CUDA (Google Colab Edition)

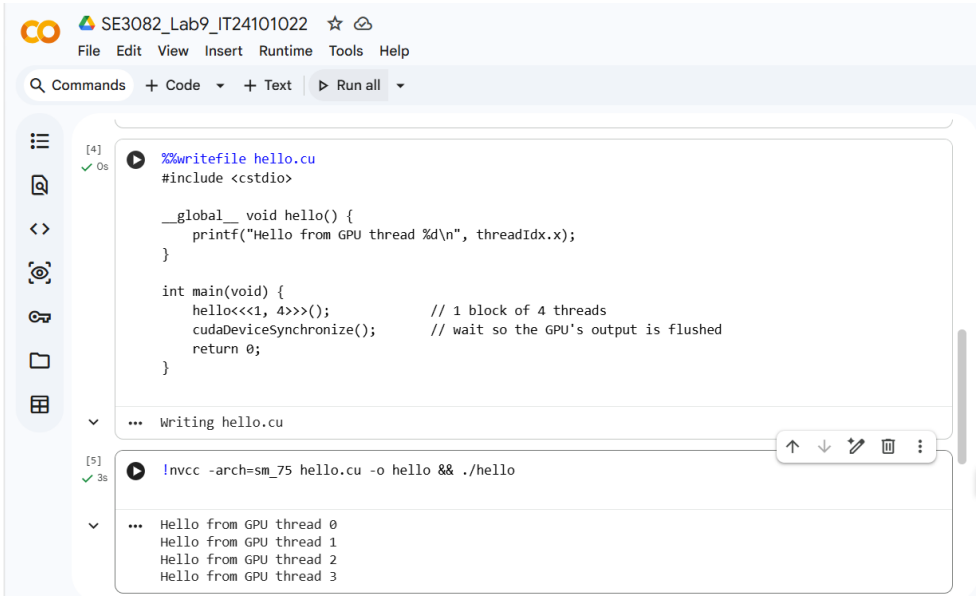
SE3082 – Parallel Computing | Semester 2, 2026

- **IT24101022 – Mannapperuma.M.A.S.K.M.**

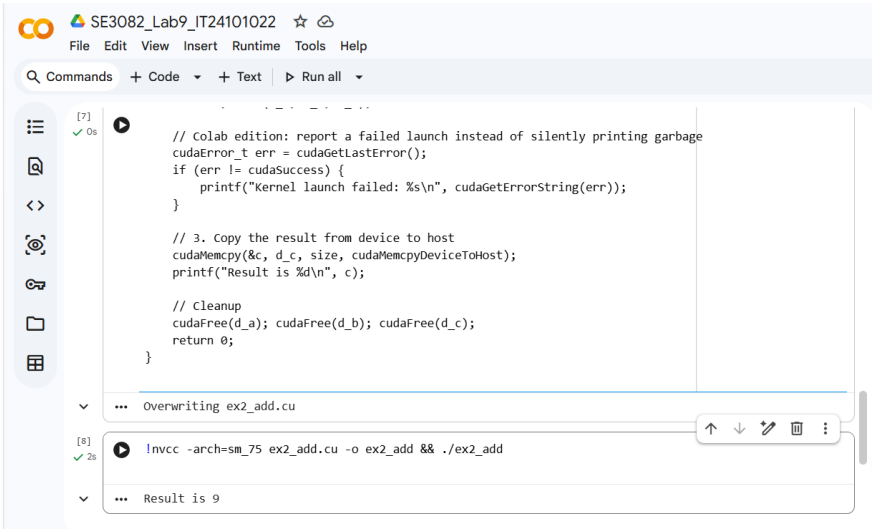
Exercise – 01




SE3082_Lab9_IT24101022
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
|===+========================+=======
| 0 Tesla T4 Off | 00000000:00:04.0 Off |
| N/A 44C P8 12W / 70W | 0MiB / 15360MiB | 0
+---+------------------------+-------
+---
| Processes:
| GPU GI CI PID Type Process name
| ID ID
|===
| No running processes found
+---
[2]
0s
!nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_01:58:59_PM_PDT_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0


SE3082_Lab9_IT24101022
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[4]
0s
%%writefile hello.cu
#include <cstdio>
__global__ void hello() {
printf("Hello from GPU thread %d\n", threadIdx.x);
}
int main(void) {
hello<<<1, 4>>>(); // 1 block of 4 threads
cudaDeviceSynchronize(); // wait so the GPU's output is flushed
return 0;
}
Writing hello.cu
[5]
3s
!nvcc -arch=sm_75 hello.cu -o hello && ./hello
Hello from GPU thread 0
Hello from GPU thread 1
Hello from GPU thread 2
Hello from GPU thread 3

Exercise – 02



THINK

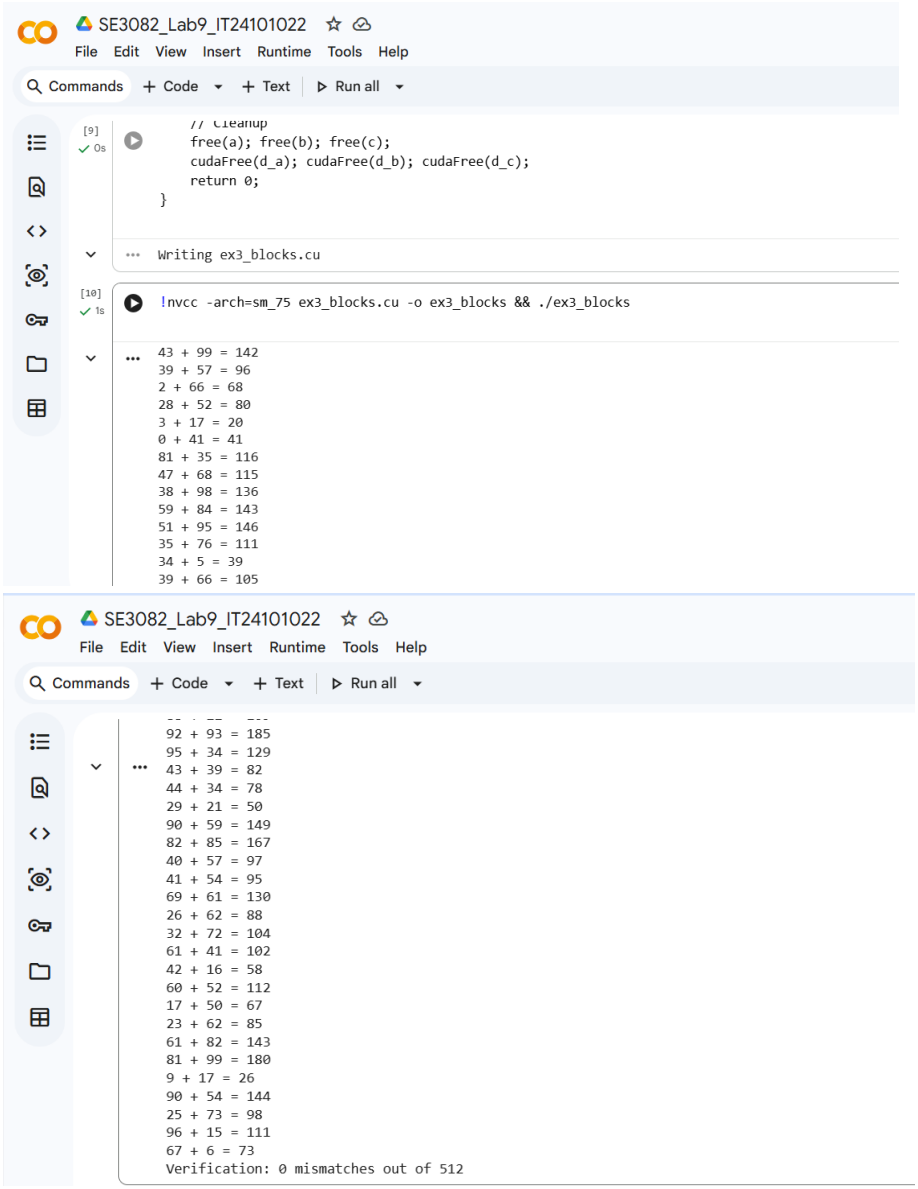
1) The CPU (Host) and GPU (Device) have physically separate memory spaces. Variables a, b, and c are stored in the host system's RAM, which the GPU cores cannot directly access. Therefore, dedicated device memory must be allocated on the GPU VRAM using cudaMalloc (referenced by pointers d_a, d_b, d_c), and data must be explicitly transferred between the two memories using cudaMemcpy.

2) "a" is a local integer variable on the host stack, so we need to pass its address (&a) to cudaMemcpy so the function knows where to read the host bytes from.
In contrast, d_a is already a pointer holding the memory address allocated on the GPU device (returned by cudaMalloc). When launching the kernel add<<<1, 1>>>(d_a, ...), the kernel expects a pointer to device memory (int *a), so passing d_a directly provides that exact address.

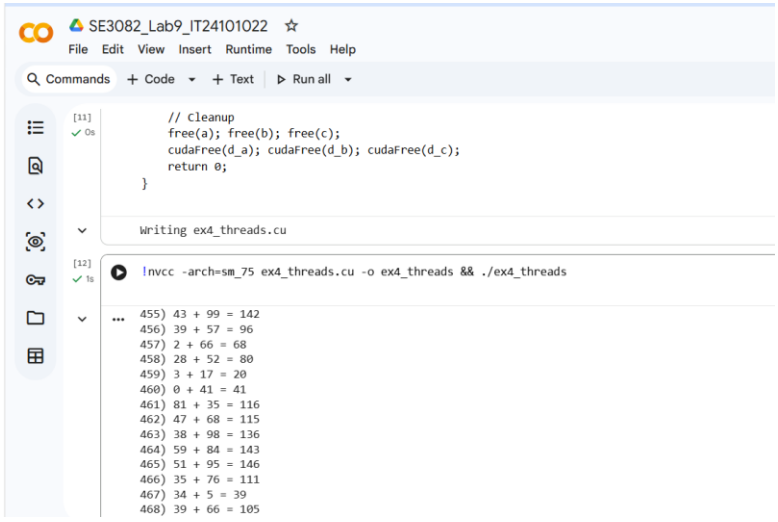
3) Current threads: Exactly 1 thread runs the kernel (configured as add<<<1, 1>>>, which means 1 block with 1 thread).
To run with 10 threads: Change the kernel launch configuration to add<<<1, 10>>>(d_a, d_b, d_c); (1 block containing 10 threads).
Would the answer change? No, the numerical answer would remain 9. However, all 10 threads would execute concurrently and perform the identical operation (*c = *a

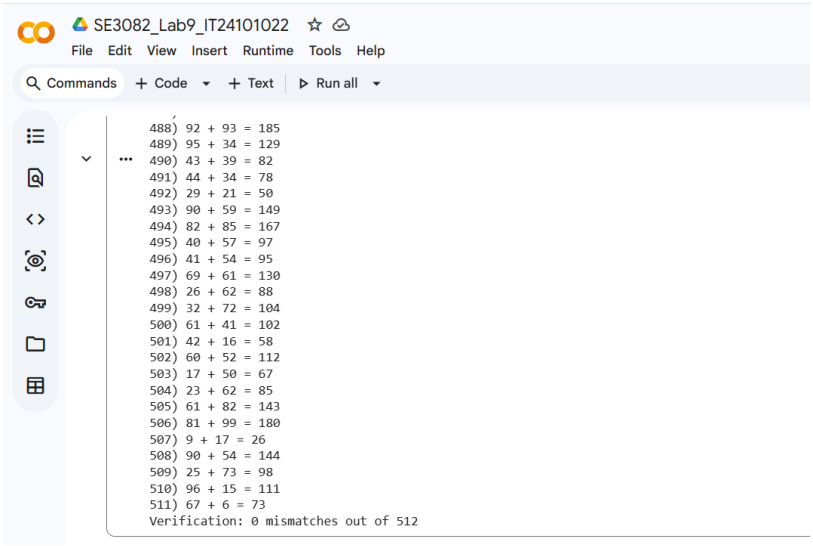
+ *b), causing a race condition (write collision) where all 10 threads attempt to write the value 9 to the exact same device memory location *c at the same time.

Exercise – 03



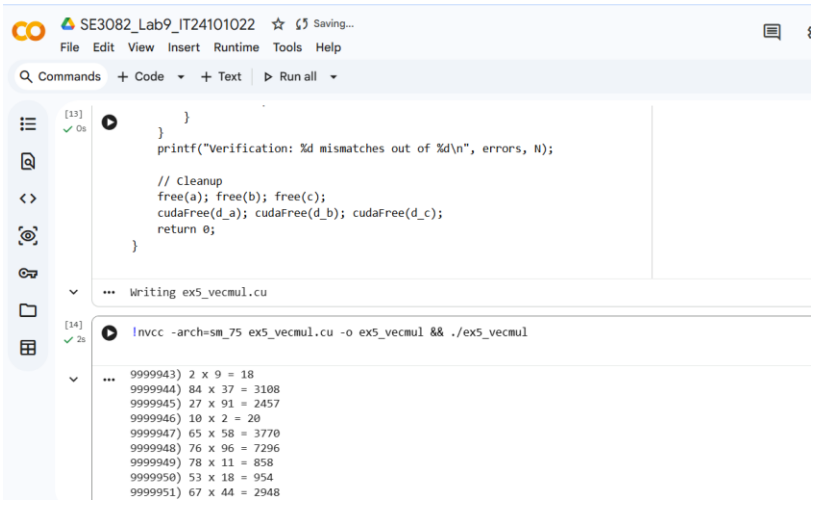
Exercise – 04

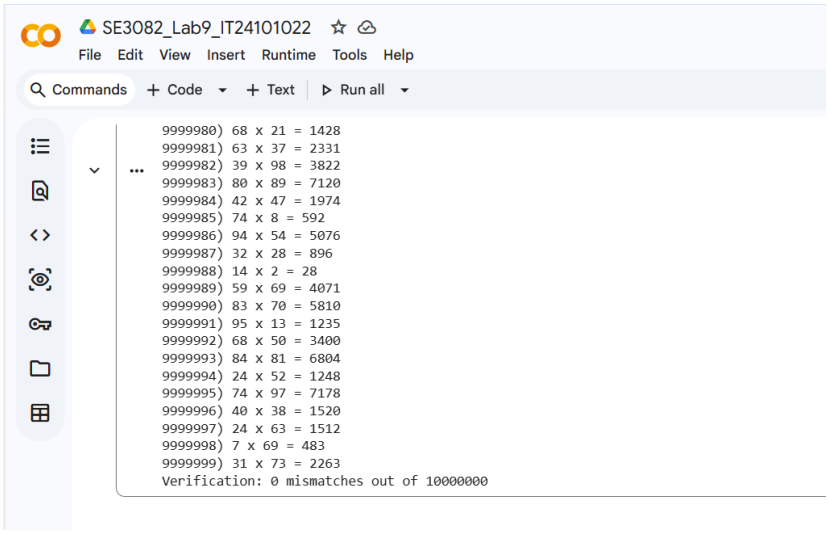




What is printed: Kernel launch failed: invalid configuration argument
Which part told you: The error check using cudaGetLastError() right after the kernel launch.
Which limit of the GPU did you hit: The maximum number of threads per block limit. On NVIDIA CUDA hardware (including the T4), a single block can have at most 1024 threads. Setting $N = 2048$ exceeds this hardware limit, causing the kernel launch to fail with an invalid configuration argument. (Note: Remember to keep/change it back to #define N 512)

Exercise – 05





THINK

Why is it a good idea to use a multiple of 32 for the threads per block? What happens to the total number of
threads launched if you use 500 instead of 512?

Why multiple of 32: GPUs execute threads physically in hardware groups called warps (which always consist of 32 threads). When threads per block is an exact multiple of 32 (e.g., 512 = 16×32), all warps are fully populated, ensuring 100% warp utilization and maximum hardware performance.
If using 500 instead of 512: 500 is not divisible by 32 ($500 / 32 = 15.625 \rightarrow 16\text{ warps}$). The 16th warp would only have 20 active threads,

leaving 12 threads idle in every single block, wasting hardware execution slots. Additionally, calculating blocks needed for 10M elements with 500 threads per block gives 20,000 blocks ($20,000 \times 500 = 10,000,000$ threads), which launches more blocks overall.

Exercise – 06

